Search Scaling

reads scale with copies, writes with shards

Watch the 4-min video

In one line: every search is **scatter-gather** across shards, so the slowest shard sets latency; scale **reads** with replicas, routing, partitioning and caching, and scale **writes** with more primaries, bulk requests and longer refresh intervals, because every write lands on a primary and all of its replicas.

1 The campaign spike

A cluster serves 500 queries per second at 40 ms. Marketing launches a campaign; traffic jumps to 10,000 QPS and latency climbs to 2 seconds, while a nightly bulk import is hammering the same nodes. Search reads and writes compete for the same CPU, disk and page cache, and they scale with different levers. Knowing which lever helps which side is the whole lesson.

2 How a query runs: scatter-gather

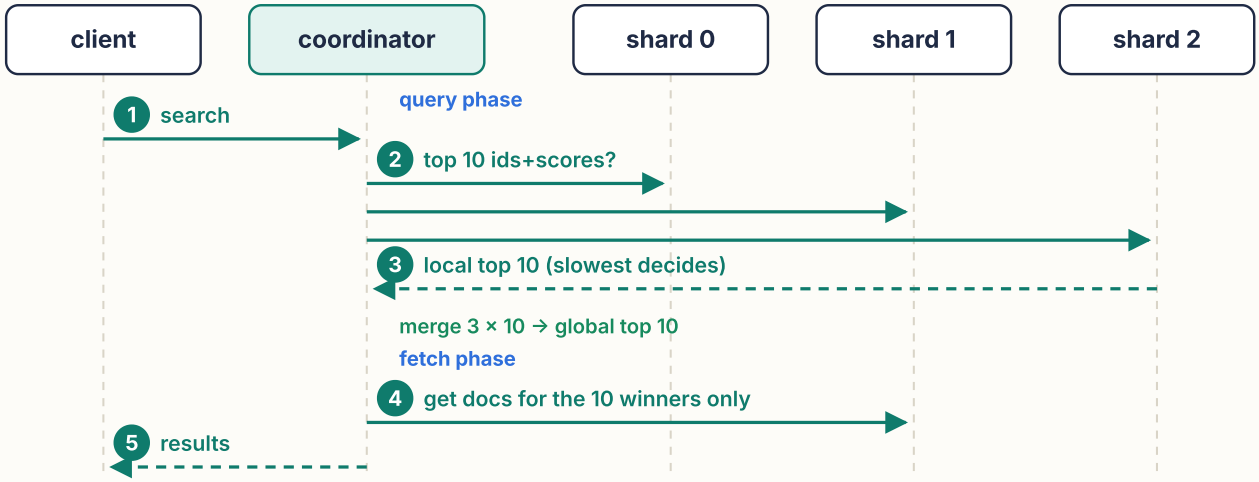


Figure 1. Elasticsearch's default query_then_fetch. Each shard returns only IDs and scores; documents are fetched for the final winners.

The coordinating node sends the query to **one copy of every shard** of the target indices. Each shard scores locally and returns its top `from + size` IDs and scores; the coordinator merges them, then fetches the full documents for the winners only. Three consequences:

- **The slowest shard decides.** A query is as slow as its slowest shard copy: a GC pause, a merge or a hot node on any one of them delays the whole response.
- **Fan-out amplifies tails.** If 1% of shard requests are slow, a query touching 20 shards is slow 18% of the time.
- **Deep pagination is expensive.** Page 1,000 (`from = 10,000`) makes every shard return 10,010 hits and the coordinator sort 20 × 10,010. Elasticsearch caps `from + size` at 10,000 (`index.max_result_window`); use `search_after` with a point-in-time for deep scrolling.

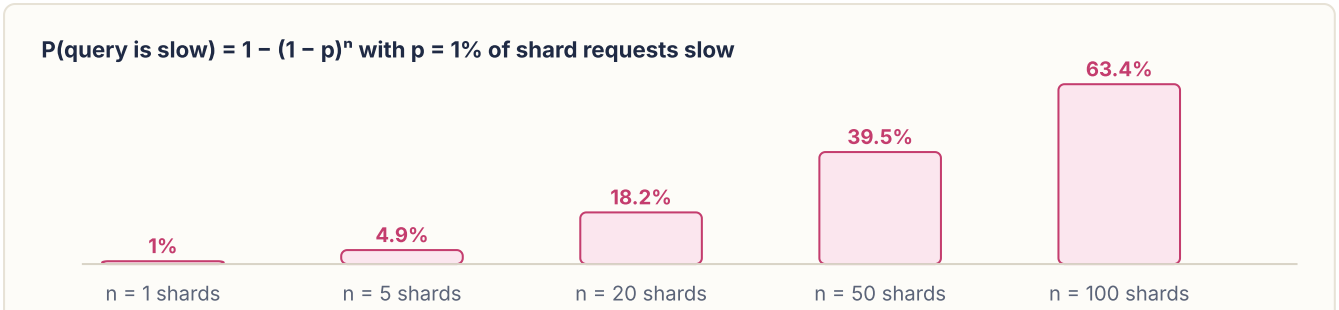


Figure 2. Tail amplification with fan-out ('The Tail at Scale', Dean and Barroso). Fewer shards per query means fewer chances to hit a slow one.

3 Scaling reads

- **Replicas**: each replica is a full copy that can answer queries, so 1 primary + 3 replicas gives roughly 4× the query capacity *if* the extra copies sit on extra nodes. Replicas on the same overloaded nodes add nothing.
- **Adaptive replica selection** (on by default since Elasticsearch 7) sends each shard request to the copy with the best recent response time and queue size, steering around slow nodes.
- **Fewer shards per query**: partitioning and routing (below). Also avoid oversharding: 100 tiny shards per query is 100 tasks of fixed overhead.
- **Memory for the page cache**: Lucene reads segments through the OS cache, so keep JVM heap ≤ 50% of RAM (lesson 16) and size nodes so the hot data fits in what's left.
- **Cheaper queries**: filters instead of scoring clauses (lesson 18), no leading wildcards, sensible aggregation sizes, `track_total_hits` left at its default.

4 Scaling writes is different

Every document is indexed on its primary and then **re-indexed on every replica** (Elasticsearch replicates documents, not files), so adding replicas makes writes **more** expensive. Write levers:

- **More primary shards** (on more nodes) spread indexing; choose at creation or via rollover (lesson 16).
- **Bulk requests**: hundreds to thousands of docs per `_bulk` call cut per-request overhead dramatically (lesson 19).
- **Longer refresh interval** (e.g. 30 s, or -1 during a one-off load): fewer tiny segments, less merging.
- **Replicas to 0 during a one-off load**, then back to 1+: replicas then copy finished segments instead of re-indexing each document.
- **Separate the work**: run imports off-peak, or on separate hot nodes. OpenSearch's **segment replication** copies segments to replicas instead of re-indexing, cutting replica CPU; Elastic's serverless offering separates indexing and search tiers entirely.

Lever	Reads	Writes
Add replicas (on new nodes)	↑ capacity, ↑ availability	↓ every write done N+1 times
Add primaries	more fan-out per query	↑ parallel indexing
Routing / partitioning	↑ fewer shards per query	neutral (watch hot shards)
Longer refresh interval	results less fresh	↑ fewer segments, less merging
Caching	↑ repeated queries skip the engine	refreshes invalidate shard caches

5 Partitioning and routing

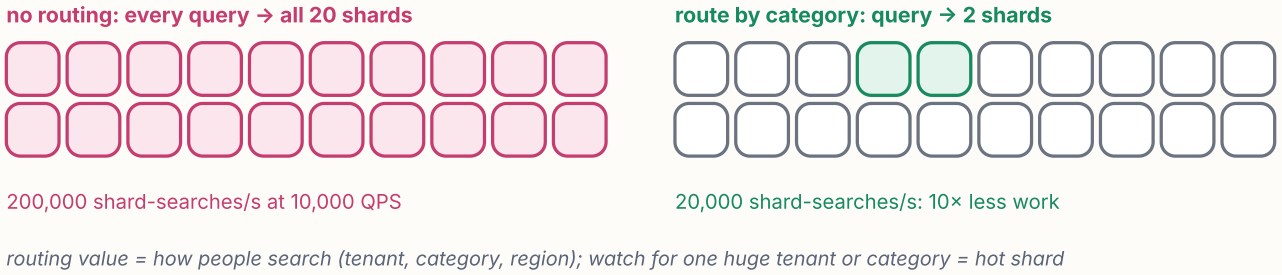


Figure 3. Routing by a key that matches how people search turns a 20-shard fan-out into a 2-shard lookup.

Time-based indices (lesson 16): a query for the last 24 hours touches today's index, not a year of data. **Custom routing**: index each document with `routing = tenant_id` (or category) and pass the same value at query time; the query goes only to the shard(s) holding that key. `index.routing_partition_size` spreads one key over several shards to soften hot spots. **Per-tenant indices** isolate big tenants entirely (lesson 41) but multiply shard counts if overused. Routing only pays off when most queries carry the routing key; a query without it still has to visit every shard.

6 Caching

Inside the engine: the **node query cache** for filters, the **shard request cache** for aggregation-only results, and the OS page cache for segments (lesson 18). In front of it: an application cache (Redis) or CDN for the most popular queries, keyed by the normalised query, filters and page. Search traffic is usually skewed, so a small set of head queries can be a large share of requests, but measure the hit rate: personalised or long-tail traffic may hit rarely, and a 5% hit rate only adds complexity and staleness.

WORKED EXAMPLE: 10,000 QPS AT ABOUT 50 MS

The product index has 20 primaries. Before: 1 replica, every query touches all 20 shards, so 10,000 QPS means **200,000 shard-searches/s**. Changes: (1) **routing by category**, since almost every product search is within a category; most queries now touch 2 shards, so 20,000 shard-searches/s. (2) **Replicas 1 → 3** on twice as many nodes, giving 4 copies to spread that load. (3) The **top 1,000 queries cached for 60 s** in Redis; if, say, 30% of traffic hits, the engine sees ~7,000 QPS. (4) **Bulk imports** move to night with `refresh_interval: 30s`, so daytime nodes only serve searches. Load tests at 10,000 QPS now show p50 around 50 ms with headroom; the remaining p99 spikes trace to merges, which the night-time imports also reduce.

TRADE-OFFS

- **More replicas**: read capacity and resilience vs write cost and disk.
- **More shards**: write parallelism vs per-query fan-out and tail latency.
- **Routing**: far less work per query vs hot shards from skewed keys.
- **Result caching**: big savings for head queries vs staleness and invalidation.

COMMON MISTAKES

- Adding replicas to fix slow indexing, which makes it slower.
- Adding replicas without adding nodes, so the same hardware just holds more copies.
- Hundreds of tiny shards per query: fan-out and overhead, not parallelism.
- Allowing unlimited deep pagination instead of search_after.

7 Key terms

Scatter-gather: fan the query out to every shard, merge results

Adaptive replica selection: route to the fastest copy

Tail amplification: more fan-out, more chance of a slow part

query_then_fetch: IDs and scores first, documents for winners

Custom routing: send a key's docs and queries to one shard

search_after: cursor-style deep pagination

8 Interview questions

Q1. How do you scale search reads versus writes? BASIC

Reads scale with replicas on more nodes, routing and caching, because any copy can answer; writes scale with more primaries, bulk indexing and longer refresh intervals, because every write goes to the primary and all replicas.

Q2. Why does partitioning help? BASIC

Queries touch fewer shards, so there is less scatter-gather work per request and fewer chances of hitting a slow shard.

Q3. p50 is fine but p99 is terrible on a 50-shard index. What do you look at? SCENARIO

Fan-out tail amplification: slow nodes (GC, merges, hot shards), adaptive replica selection, and reducing shards per query via routing, time-based indices or fewer, larger shards.

Q4. Indexing throughput dropped after you went from 1 to 3 replicas. Why? SCENARIO

Each document is now indexed on four copies instead of two; for bulk loads drop replicas temporarily, or consider segment replication.

Q5. Users want to page to result 50,000. How? SCENARIO

Not with from/size (capped at 10,000 and expensive); use search_after with a point-in-time, or redesign the UX with filters.

Q6. Would adding more shards make a slow query faster? TRICKY

Only if shards are huge and CPU is idle; otherwise more shards mean more fan-out, more overhead and worse tails. Measure per-shard query time first.

Q7. When does custom routing hurt? TRICKY

When keys are skewed (one giant tenant makes a hot shard) or many queries lack the key and must still hit every shard.

Next: SD-Q-001 · Real-Time Messaging backendroadmap.com